ECE341 - Homework 9

Ton-Minh-Ky Tran

April 18, 2026

1 Display switches states to LED row

Trivial. Using the fact that the LED is memory-mapped to `0x8000000c` and the switches are to `0x80000010`, we load the half-word from the latter and store it back to the former. An implementation of the program is located in `24125102_01/Prog1.txt`.

2 Toggle an LED with a button

Same premise as Exercise 1, with the buttons are memory-mapped to the bits 10-12 of the word stored in `0x80000010`. The only tricky part is that we require the button `B0` to toggle the LED per press instead of per clock cycle when it's enabled. In other words, we wish for the toggle to be edge-triggered instead of level-triggered, which can easily be dealt with by storing the previous buttons states. The toggle can be trivially implemented by taking the XOR of the current LED state and `0x1`. An implementation of the program is located in `24125102_02/Prog2.txt`.

3 LED command panel using buttons

Same premise as Exercise 2, with different operations—shifting left by one bit via `sll` and inverting via `nor`. An implementation of the program is located in `24125102_03/Prog3.txt`.

4 Counter stored in RAM and displayed via LED

Simply set `0x0` as the memory address of the current counter value, and with the same techniques as devised in Exercise 2 and 3, we simply pull the value, update it, and push the value to LED. An implementation of the program is located in `24125102_04/Prog4.txt`.

5 Counter stored in RAM and displayed via 7-segment display

By modifying the southbridge to append a peripheral 7-segment display that utilizes 8 bits, we use the same solution to Exercise 4—with a notable difference that we need to wrap the value in case of over- or underflow. An implementation of the program is located in `24125102_05/Prog5.txt`.

6 Display symbols on 8×8 display

We first modify the southbridge to append a peripheral 8×8 display. The display is split into two top and bottom halves—the former is memory-mapped to `0x80000018` and the latter is to `0x8000001c`. Preloading the RAM with the symbol data that determines which pixels to light up to draw the symbol, we simply load the required data to the display and change the data to be loaded whenever `B1` is pressed. An implementation of the program is located in `24125102_06/Prog6.txt`, and the RAM image is located in `24125102_06/Prog6_data.txt`.